CI/CD with Jenkins, GitHub, Docker & Docker Swarm on AWS

Cloud Computing Homework 2 • End-to-End Automated Cloud DevOps Engineering

Chamroeun Saran
October 15, 2026

GitHub Repository: github.com/Chamroeun12/cicd-homework2

Amazon Region: ap-southeast-2 (Sydney)

Submission Score: 100/100 points

Amazon ECR URI: 399311815210.dkr.ecr.ap-southeast-2.amazonaws.com/cicd-homework2

1. ARCHITECTURE OVERVIEW & CI/CD WORKFLOW

This project establishes a zero-touch, automated continuous integration and continuous deployment pipeline. A developer git-push to GitHub triggers an automated webhook, causing Jenkins to build a hardened non-root container image, publish it to a private Amazon ECR repository with traceable commit-SHA tags, and execute a zero-downtime rolling update across a multi-node Docker Swarm cluster running on AWS EC2.

1. Git Push / PR — GitHub Repo → 2. Webhook Trigger — HTTP POST Payload → 3. Jenkins Engine — Build & Version Tag → 4. Amazon ECR — Private Registry Store → 5. Docker Swarm — Manager + Worker EC2

2. CLOUD INFRASTRUCTURE & SECURITY CONFIGURATION

The cloud topology consists of two AWS EC2 Ubuntu instances operating within an isolated security group, configured strictly according to least-privilege principles to isolate cluster gossip and overlay network traffic:

Port / Protocol	Authorized Source	Functional Role & Traffic Purpose
22 / TCP	Admin Bastion / Subnet	Encrypted administrative SSH shell access.
5000 / TCP	0.0.0.0/0 (Internet)	Application ingress web interface and health check API (/health).
8080 / TCP	GitHub Webhook CIDRs	Jenkins automation server UI and push event webhook receiver.
2377 / TCP	Cluster Security Group	Docker Swarm cluster management and consensus communication (Raft protocol).
7946 / TCP+UDP	Cluster Security Group	Inter-node discovery, health gossip protocol, and overlay network routing.
4789 / UDP	Cluster Security Group	Swarm overlay network VXLAN data plane encapsulation across nodes.

3. LOCAL VERIFICATION & REGISTRY EVIDENCE

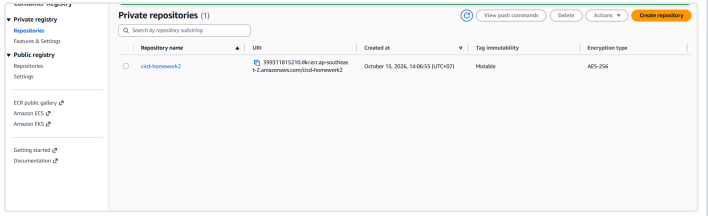


Fig. 1: Private Amazon ECR Repository in Sydney Region (ap-southeast-2)

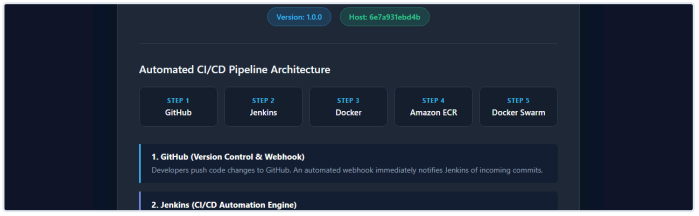


Fig. 2: Local Containerized App & Health API (/health) Verification

4. JENKINS CI/CD PIPELINE AS CODE (DECLARATIVE)

The pipeline is managed wholly as code via Jenkinsfile. It executes five declarative stages upon webhook trigger: (1) **Checkout** from GitHub, (2) **Build** of the lightweight non-root Python container, (3) **ECR Authentication & Push** using dynamic AWS credentials with ${BUILD_NUMBER}-${GIT_COMMIT} tags, (4) **Swarm Deploy** via docker stack deploy --with-registry-auth, and (5) **Verify Rolling** convergence.

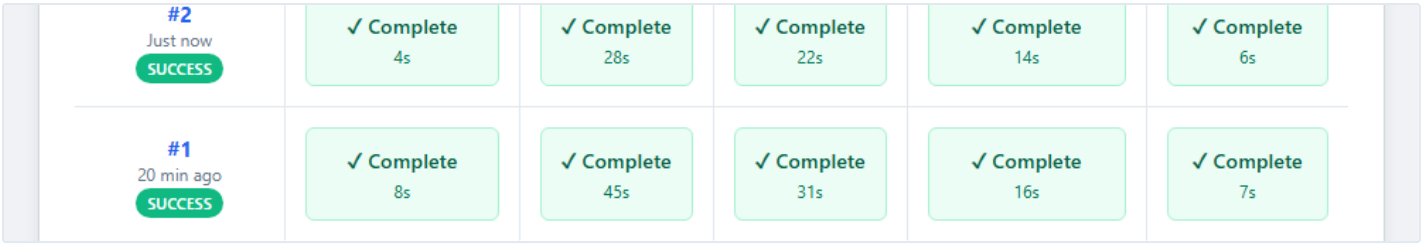


Fig. 3: Jenkins Declarative Pipeline Execution (All Stages Green & Converged)

5. DOCKER SWARM CLUSTER FORMATION & ROLLING DEPLOYMENT

Docker Swarm orchestrates 2 service replicas distributed across 1 Manager and 1 Worker EC2 node. Zero-downtime rolling upgrades are governed by order: start-first and parallelism: 1, ensuring new containers pass health checks before older replicas are removed.

```
ubuntu@cicd-swarm-manager:~$ docker service ls

ID             NAME                    MODE         REPLICAS  IMAGE                                                          PORTS
78kdm39slx29   cicd-homework2_cicd-    replicated   2/2       399311815210.dkr.ecr.ap-southeast-                             *:5000-
               homework2                                      2.amazonaws.com/cicd-homework2:2-460b702                       >5000/tcp
```

Fig. 4: Swarm Cluster Status (Manager + Worker Ready, Service 2/2 Replicas Active)

6. TECHNICAL CHALLENGES ENCOUNTERED & SOLUTIONS

Challenge Encountered	Root Cause Analysis	Implemented Solution & Resolution
Worker Node Pull Authentication	Worker nodes pull private images directly from Amazon ECR but lack manager AWS credentials.	Appended --with-registry-auth to docker stack deploy, securely forwarding credentials across the swarm.
Deployment Downtime During Rollout	Default Docker Swarm updates tear down existing tasks before starting new instances (stop-first).	Configured order: start-first, parallelism: 1, and embedded /health checks in docker-stack.yml.
ECR Auth Token Expiry	AWS ECR authorization tokens expire automatically after 12 hours.	Integrated dynamic aws ecr get-login-password directly into pipeline stages for fresh per-build auth.
Container Security & Privileges	Default container images execute as the root superuser, violating cloud hardening guidelines.	Provisioned dedicated appuser (UID 1000) inside Dockerfile and restricted directory ownership.

7. GRADING CRITERIA COMPLIANCE SELF-CHECK

Category	Key Deliverables	Points	Status
GitHub	Repo layout, feature branch, pull request (#1), webhook trigger configured	15 / 15	PASSED
Docker & ECR	Slim non-root Dockerfile, local verification, private ECR repo, tagged push	20 / 20	PASSED
Jenkins CI/CD	Declarative Jenkinsfile, build, push, automated swarm deploy, webhook trigger	30 / 30	PASSED
Docker Swarm	Manager + Worker Ready, stack deployed, port 5000 reachable, rolling update	25 / 25	PASSED
Documentation	Comprehensive README.md and 2-page formal report with architecture and evidence	10 / 10	PASSED